

AI Software Code Reviewer & Secure Development Agent

A multi-agent pipeline that finds bugs & vulnerabilities,
suggests grounded fixes, verifies them, and asks a human before applying.

LangGraph

Ollama llama3.2:1b

Semgrep

RAG: FAISS / TF-IDF

Streamlit

Team Alpha • Hackathon PS #06

Architecture & Working — in 13 simple slides

1. What it does

Automating secure code review, end to end

The problem

- Manual code review is slow and misses security bugs.
- Static tools flag issues but don't explain or fix them.
- AI fixes can hallucinate unsafe 'best practices'.

Our solution — one automated pipeline that:



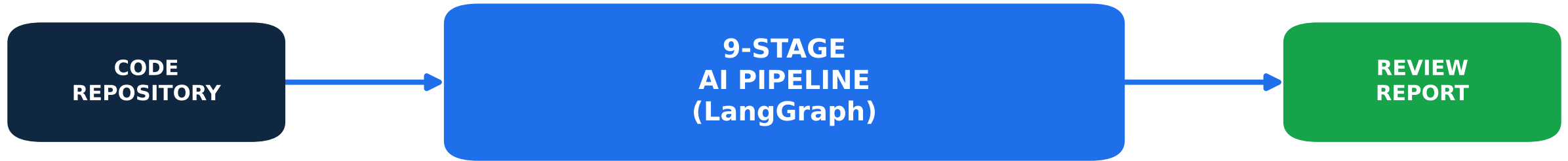
Input: a code repository

→

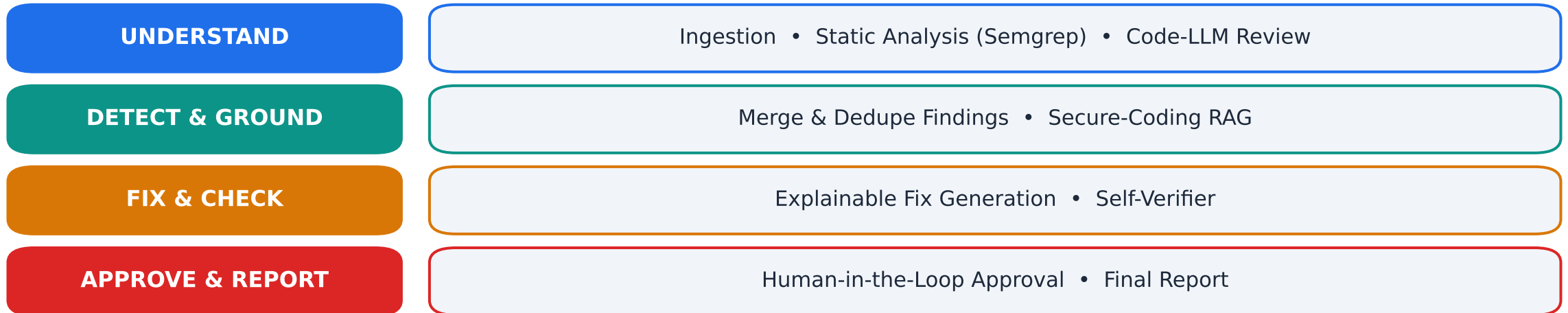
Output: an explainable review report + verified fixes

2. The big picture

9 specialist agents on a LangGraph pipeline

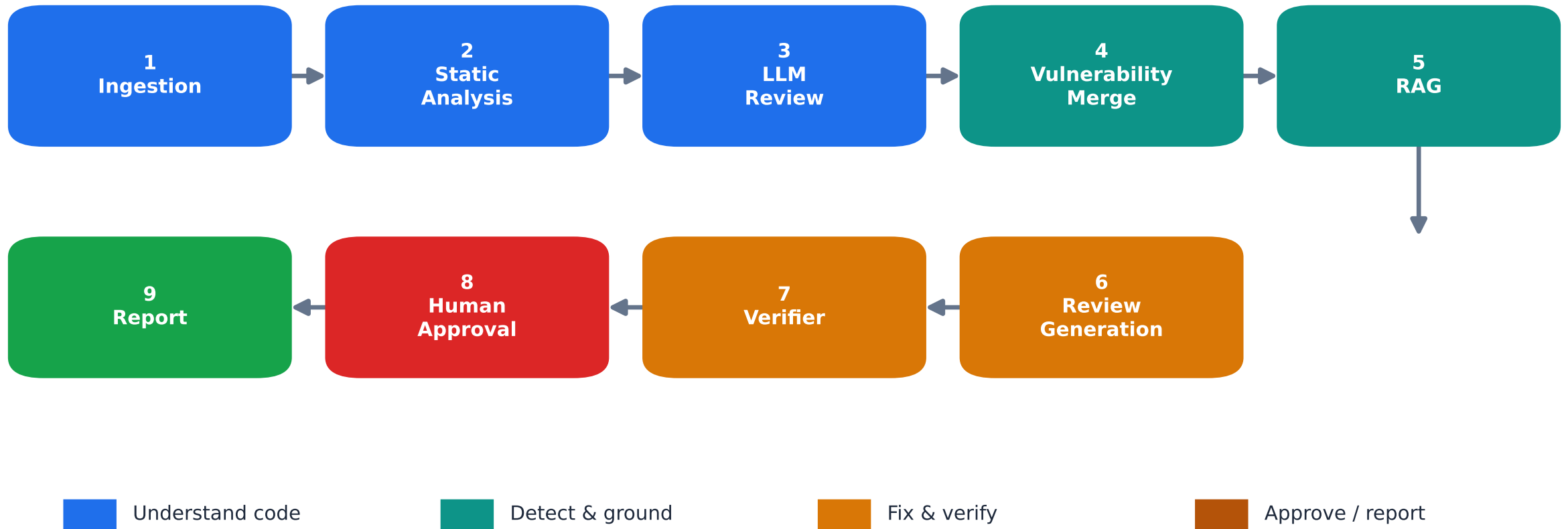


Every stage adds to one shared 'ReviewState' that flows left to right



3. The pipeline flow

Data moves through 9 stages, in order



Linear, easy to trace: each stage reads the previous stage's output.

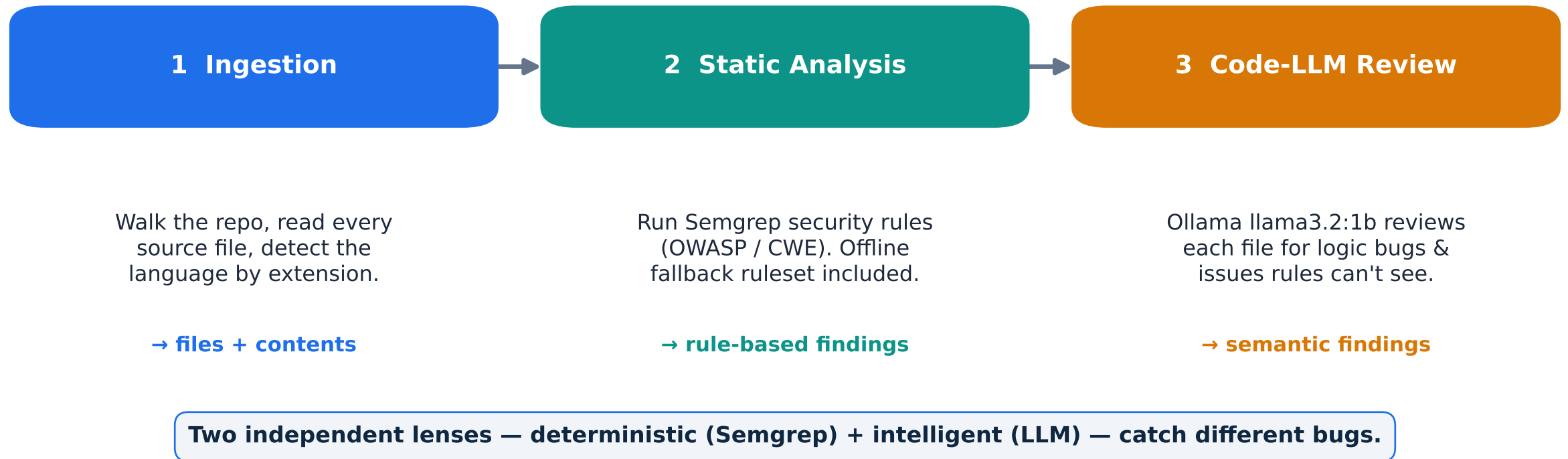
4. The 9 agents at a glance

Each LangGraph node is one specialist agent (agents/*.py)

1	Ingestion	Discovers files, reads contents, detects language. agents/ingestion_agent.py
2	Static Analysis	Runs Semgrep security rulesets + builds a code graph. agents/static_analysis_agent.py
3	LLM Review	Code-LLM reviews each file for logic & quality bugs. agents/llm_review_agent.py
4	Vulnerability	Merges & dedupes Semgrep + LLM findings; splits security vs quality. agents/vulnerability_agent.py
5	RAG	Retrieves grounded secure-coding guidance for each finding. agents/rag_agent.py
6	Review Generation	Writes an explainable comment + a minimal, grounded fix. agents/review_generation_agent.py
7	Verifier	Syntax check, no-op check, Semgrep regression re-scan. agents/verifier_agent.py
8	Approval	Routes CRITICAL/ERROR or flagged fixes to a human. agents/approval_agent.py
9	Report	Assembles all artifacts as Markdown + JSON. agents/report_agent.py

5. Understanding the code

Stages 1-3: read → scan → reason



6. Detect & ground the findings

Stages 4-5: merge, then retrieve guidance



same file+line = one finding

Then split:

Security vulnerabilities

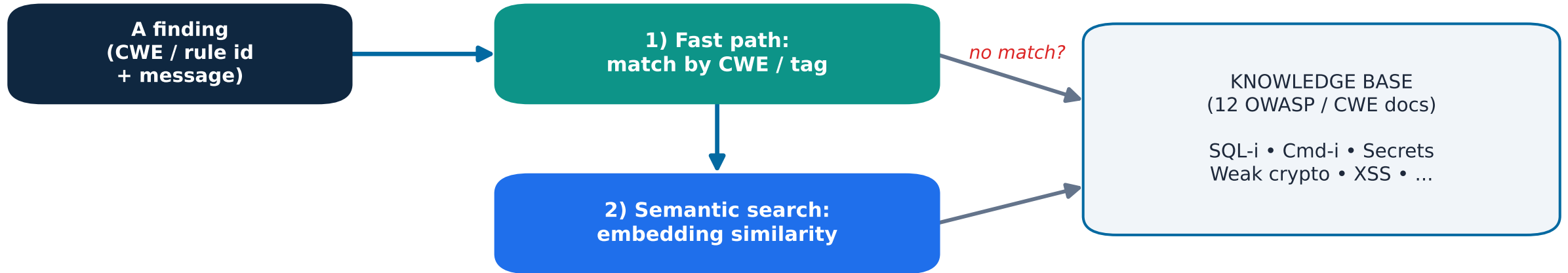
Quality issues

RAG = Retrieval-Augmented Generation

For each finding, pull the matching secure-coding guidance from a knowledge base so the fix is grounded in real documentation — not guessed.

7. Inside the RAG engine

How grounded guidance is retrieved



Two interchangeable backends:

Primary: sentence-transformers + FAISS

Fallback: scikit-learn TF-IDF (offline)

If embedding models can't download, it auto-switches to TF-IDF — RAG still works with no internet.

Retrieved snippets are injected into the fix prompt → the LLM must ground its fix in them.

8. Fix & self-check

Stages 6–7: generate, then verify

6 REVIEW GENERATION

- Plain-language explanation (why it matters)
- A minimal, targeted code fix
- Grounded in the retrieved RAG guidance

7 VERIFIER (self-reflection)

- Syntax check — does the fix parse?
- No-op check — did it actually change code?
- Regression re-scan — Semgrep rule gone?

Each fix gets a confidence score

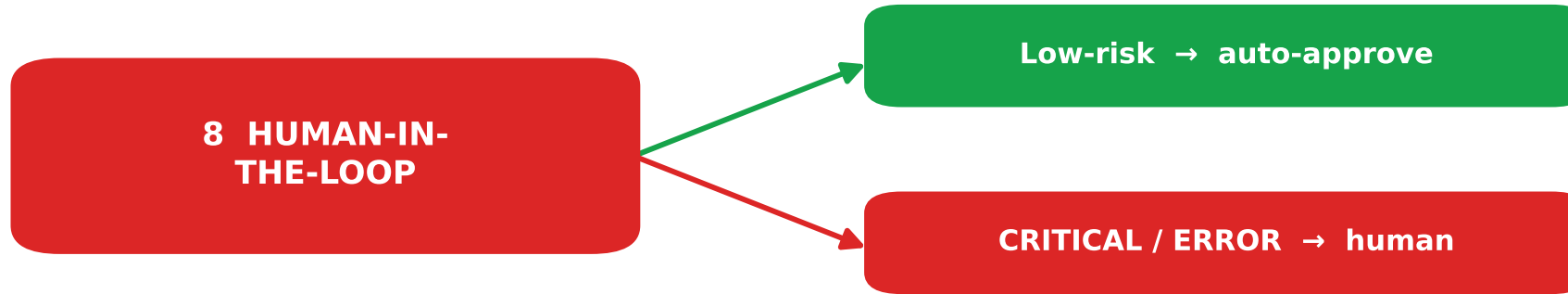
Passes → ready

Fails → flagged

The AI checks its own work before any fix is considered applicable.

9. Human gate & final report

Stages 8-9: approve, then deliver



Nothing high-risk is ever applied automatically.

9 FINAL REPORT (Markdown + JSON)

- Bug summary
- Suggested fixes
- Security report
- Confidence score
- Quality score
- Approval log

10. Tech stack & how to run

Local-first, no API key required

Built with

Orchestration

LangGraph StateGraph

Static analysis

Semgrep (OWASP/CWE)

Interfaces

CLI + Streamlit app

Code-LLM

Ollama - llama3.2:1b

RAG

FAISS / TF-IDF + KB

Output

Markdown + JSON

Run it

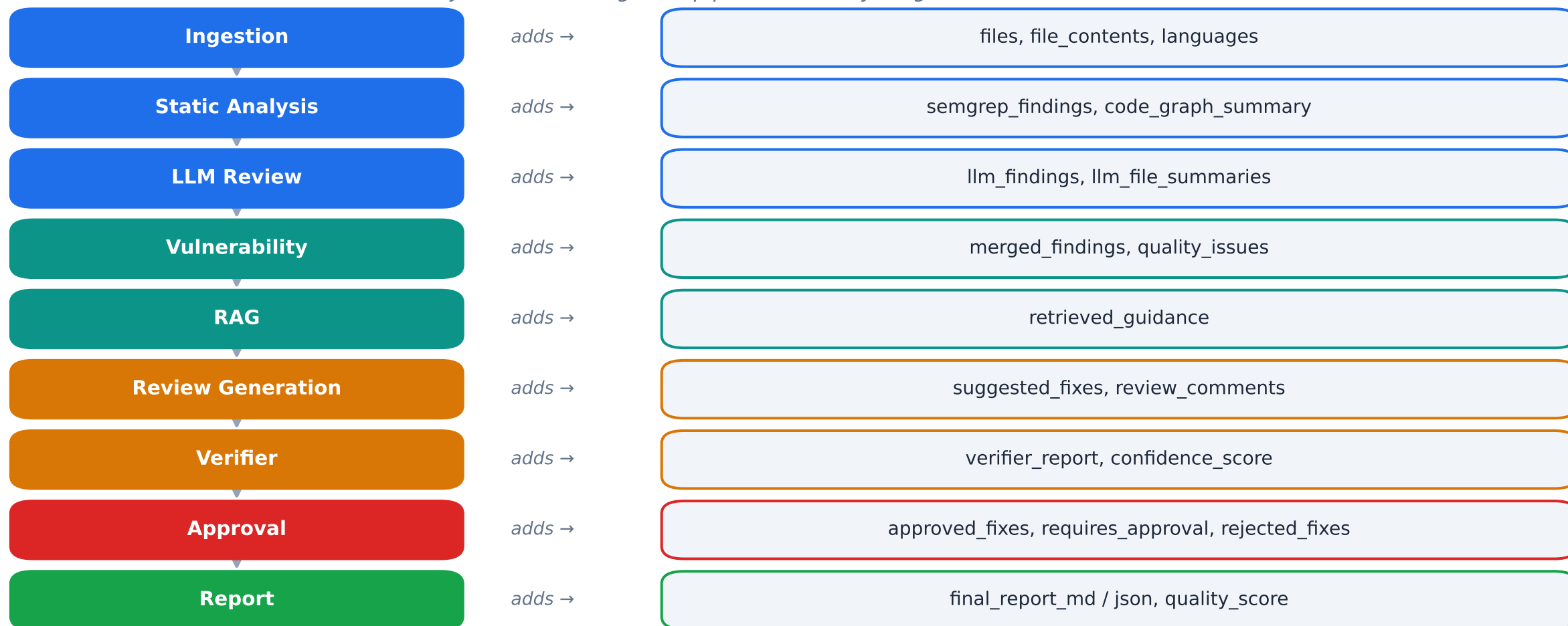
```
pip install -r requirements.txt
ollama serve && ollama pull llama3.2:1b

python main.py --repo ./sample_repo      # CLI review
streamlit run app.py                      # dashboard
python -m tests.test_pipeline             # end-to-end test
```

11. Data flow

What each stage adds to the shared ReviewState

One state object flows through the pipeline — every stage reads it and adds new fields.



Result: a fully-populated ReviewState → rendered as the final Markdown + JSON report.

12. Data sources & references

What the system reads, retrieves from, and can train on

Sources used at runtime

RAG knowledge base

rag/secure_coding_docs.json — 12 OWASP/CWE guidance docs

Semgrep rulesets

p/security-audit, p/owasp-top-ten, p/cwe-top-25, p/secrets

Offline fallback rules

rules/local_security_rules.yaml (no-internet mode)

Demo input repo

sample_repo/ — intentionally vulnerable code

Reference datasets (for training a dedicated vuln model — README §5)

Devign

C/C++ vuln detection

Big-Vul (MSR'20)

CVE-labeled vulns

CodeXGLUE

Code intelligence

CodeSearchNet

Semantic code search

Juliet (NIST)

CWE weakness cases

DiverseVul

Large-scale vulns

OWASP Benchmark

Tool benchmark

ManySSuBs4J

Java bug fixes

SAP Project-KB

Java bug benchmark

Datasets are cited as reference benchmarks; this demo grounds fixes via the RAG KB rather than training on them.